

Chapter 3 : Problem Solving and Research

Objectives:

Knowing , understanding and implementing State spaces, BFS, DFS, heuristics and A* algorithm, on Python practically.

I- Problem Solving Algorithms

Problem solving is a fundamental concept in [artificial intelligence \(AI\)](#) where systems are designed to identify challenges, make decisions and find efficient solutions. AI uses agents which are systems that perceive their environment and take actions to achieve specific goals. They go beyond simple reflex agents which react to stimuli based on pre-defined rules. Instead, problem-solving agents actively analyze situations, evaluate different options and choose the best action to reach their goal.

These agents work by:

- **Perceiving the environment:** They collect data about their surroundings such as sensor inputs or observations.
- **Defining the problem:** They clearly understand the problem including the starting point, the available actions and the desired goal.
- **Exploring different possibilities:** They consider various ways to solve the problem and evaluate which approach is likely to succeed.
- **Evaluating and deciding:** Once they explore options, they assess the outcomes and pick the best course of action based on factors like time, resources and success likelihood.
- **Learning and adapting:** Many problem-solving agents can learn from past experiences, improving their decision-making abilities over time.

Types of Problems in AI

AI systems can face different types of problems. These problems are categorized based on their impact on the system's performance and how they can be handled:

1. Ignorable Problems

These are minor issues that have little or no effect on the AI system's overall performance. They are often harmless and do not require immediate attention.

Examples:

- Small inaccuracies in predictions that don't affect the final result like a slight error in image classification.
- Minor issues in data preprocessing that don't change the outcome.

Handling: Ignored as they don't interfere with the system's primary function.

2. Recoverable Problems

Recoverable problems are those where the AI system encounters an issue but can be fixed with intervention either automatically or manually, such as error-handling functions.

Examples:

- Missing data that can be filled in using statistical methods.
- System crashes that can be fixed by restoring from a backup.

Handling: Requires action like automated error handling, retraining the model or system recovery through checkpoints.

3. Irrecoverable Problems

These are severe issues that cause permanent damage or failure, making it impossible for the system to recover. They can lead to significant performance loss.

Examples:

- Corrupted training data that causes bias and reduces the model's effectiveness.
- Adversarial attacks that make the model untrustworthy.
- Overfitting, where the model becomes too specialized and cannot adapt to new data.

Handling: Requires a complete rebuild of the system including retraining the model or updating the data to eliminate underlying issues.

Steps in Problem Solving in AI

AI problem solving follows a structured, logical process similar to human thinking, its steps include:

1. **Problem Definition:** First, the problem needs to be clearly defined. This includes understanding the inputs, constraints and what the solution should look like.
2. **Problem Analysis:** Once the problem is defined, it's analyzed in more detail. This helps in understanding its limitations and possible solutions.
3. **Knowledge Representation:** All the important information is organized so the AI can understand and work with it. This could include creating graphs or using databases.
4. **Problem Solving:** AI uses appropriate methods to solve the problem. This often means comparing different strategies to find the most efficient one.
5. **Testing and Evaluation:** Finally, after the solution is implemented, testing and evaluation ensure the solution meets all requirements and performs as expected.

Key Components of Problem Formulation in AI

Effective problem-solving in AI is dependent on several important components:

1. **Initial State:** This is the starting point of the problem where the AI begins its process. It sets the context and helps identify how the agent will approach the challenge.
2. **Action:** At this stage, AI identifies all the possible actions it can take from the initial state. Each action has an impact on how the system moves closer to solving the problem.
3. **Transition:** This refers to how the system changes from one state to another after an action is taken. Transition modeling helps show how the agent's actions influence the next steps.
4. **Goal Test:** Once an action is taken, AI checks if it has reached its goal. If the goal is achieved, the problem-solving process stops and the solution is considered complete.
5. **Cost Function:** This step assigns a numerical value to the cost of achieving the goal. The cost can include resources like time, energy or money and helps decide the most efficient way to reach the goal.

Techniques for Problem Solving in AI

Several techniques are available to solve different types of problems in AI. The choice of technique depends on the nature of the problem and the resources available.

II- Search Algorithms

Search algorithms in AI help find solutions by exploring possible paths or options in a problem space. AI uses them in tasks like pathfinding, decision making and game playing. These algorithms work by searching through a set of possibilities to reach a goal, either blindly without extra information or with guidance using heuristics.

1-State spaces

State space search in artificial intelligence is a problem-solving technique used to find a sequence of actions that transforms an initial state into a goal state. It represents all possible states of a problem and explores them systematically to identify a valid solution path. State space search is used in AI for solving puzzles, planning tasks, and decision-making problems.

Key Components of State Space Search

- **State:** A state represents a specific configuration or situation of a problem at a given point in time. Each state describes the current status of all relevant variables in the problem.
- **Initial State:** The initial state is the starting point of the search process. It defines the condition of the system before any actions are performed.
- **Goal State:** The goal state represents the desired solution or final condition that the search algorithm aims to reach. A problem is considered solved when the goal state is achieved.
- **Operators (Actions):** Operators are the valid actions or moves that can be applied to a state to generate a new state. These actions help transition from one state to another during the search process.

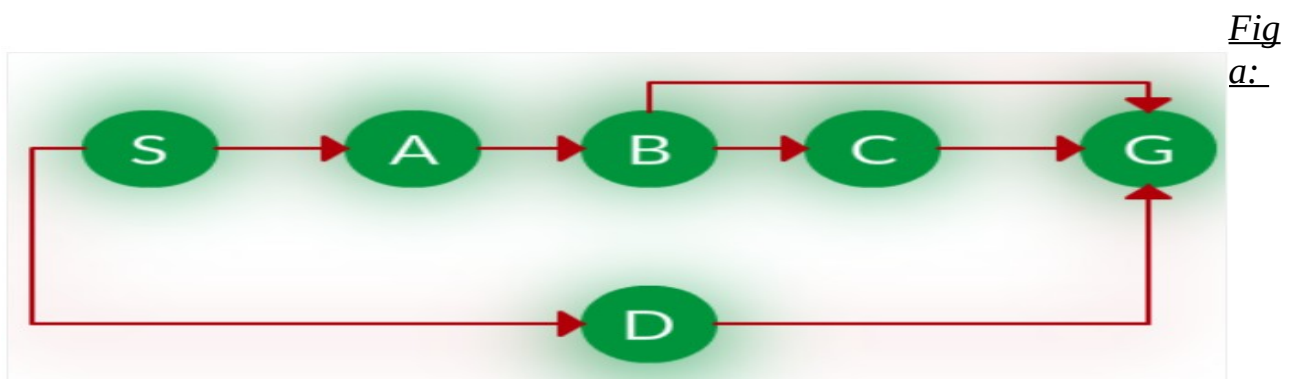
- **State Space Graph:** A state space graph is a visual or conceptual representation of all possible states of a problem and the transitions between them. In this graph, nodes represent states, and edges represent operators that connect one state to another.

2 - Uninformed searching

a - BFS

- Breadth First Search is a fundamental search algorithm that explores all possible paths level by level. It begins from the root node and explores all neighboring nodes before moving to the next level of nodes.
- BFS is complete and guarantees finding the shortest path if each move has the same cost.
- However its main drawback is high memory usage as it stores all nodes at the current level before moving deeper which can grow rapidly for large or complex problems.

For Example: Which solution would BFS find to move from node S to node G if run on the graph below.



connected graph

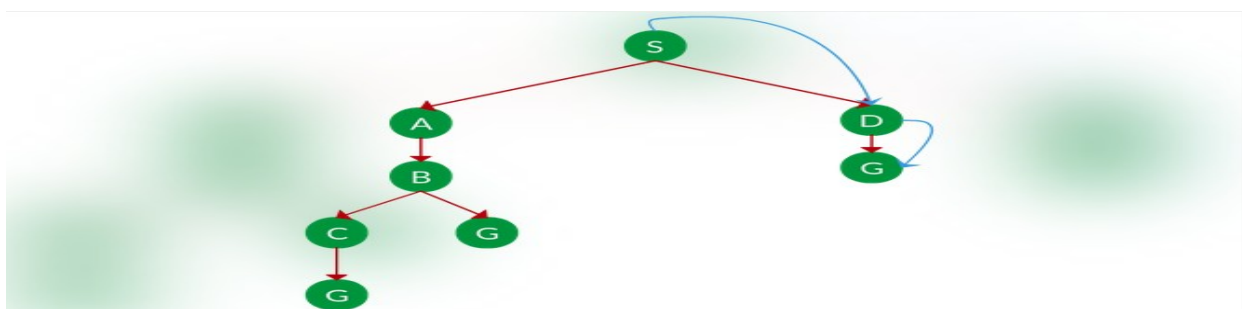


Fig b : spanning tree

As BFS traverses the tree shallowest node first it would always pick the shallower branch until it reaches the solution or it runs out of nodes and goes to the next branch.

- **Path:** S->D->G

b – DFS

Depth First Search explores paths by going as deep as possible along one direction before backtracking. It uses a stack or recursion to keep track of the path.

- DFS is memory efficient compared to BFS since it doesn't need to store all siblings at each level.
- However it is not guaranteed to find the shortest path and may get stuck in an infinite loop if the search tree is deep or contains cycles unless depth limits or visited checks are applied.

For Example: Which solution would DFS find to move from node S to node G if run on the graph below.

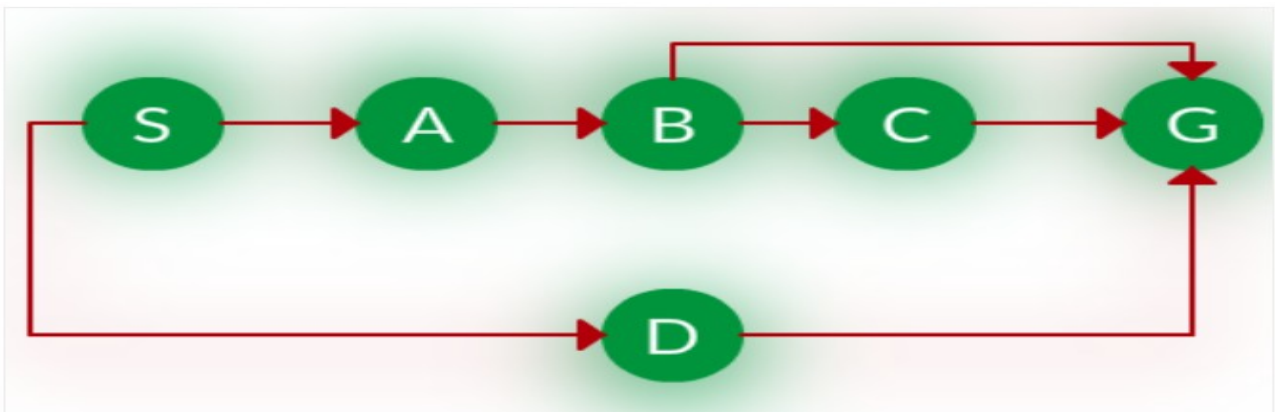


Fig c: Connected graph

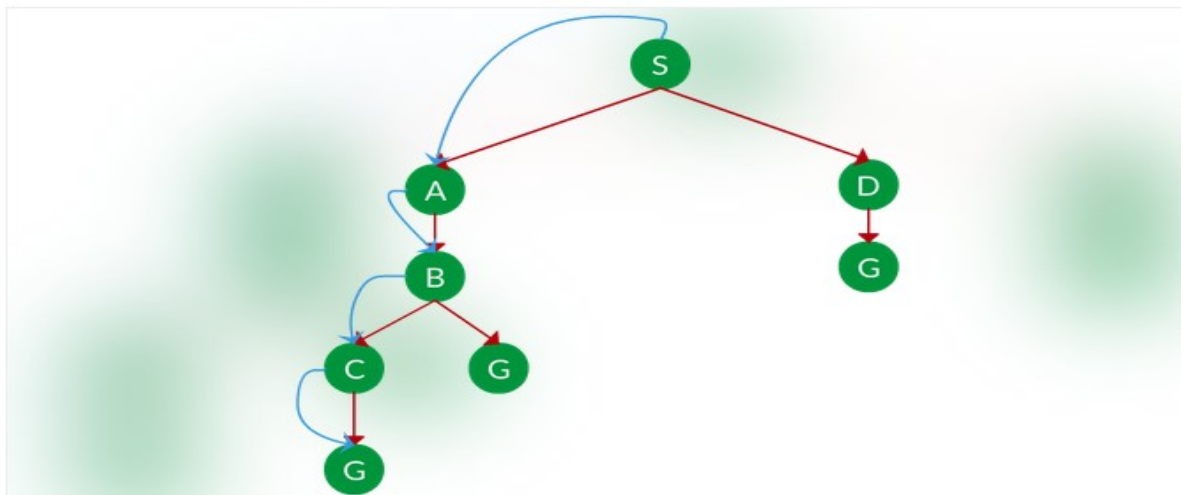


Fig d: tree

As

DFS traverses the tree deepest node first it would always pick the deeper branch until it reaches the solution or it runs out of nodes and goes to the next branch.

- **Path:** S->A->B->C->G

3 - Informed searching

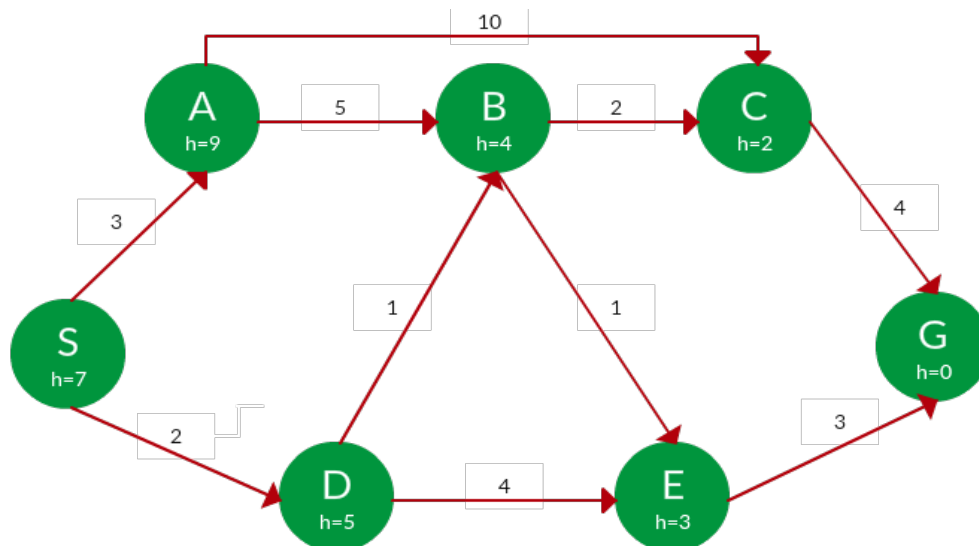
- heuristics and A* algorithm

Informed search uses domain knowledge in the form of heuristics to make smarter decisions during the search process. These heuristics estimate how close a state is to the goal guiding the search more efficiently.

A* Tree Search:

- A* tree search also uses the $f(n) = g(n) + h(n)$ evaluation but treats the search space as a tree which means it doesn't track already visited nodes. Every path is explored independently even if it leads to the same state.
- This can result in duplicate work and a larger search space, especially in graphs with cycles.
- Although it's simpler to implement A* tree search may be less efficient and is typically used when the search structure is naturally a tree or when memory constraints prevent maintaining a closed list.

For Example: Find the path to reach from S to G using A* search.



Starting from S the algorithm computes $g(x) + h(x)$ for all nodes in the fringe at each step choosing the node with the lowest sum.

The entire work is shown in the table below. It search like:

Path	$h(x)$	$g(x)$	$f(x)$
S	7	0	7
S -> A	9	3	12
S -> D	5	2	7
S -> D -> B	4	$2 + 1 = 3$	7
S -> D -> E	3	$2 + 4 = 6$	9
S -> D -> B -> C	2	$3 + 2 = 5$	7
S -> D -> B -> E	3	$3 + 1 = 4$	7
S -> D -> B -> C -> G	0	$5 + 4 = 9$	9
S -> D -> B -> E -> G	0	$4 + 3 = 7$	7
• Path: S->D->B-> G-> E and Cost: 7			

Conclusion

sources: www.google.com
www.wikipedia.org
www.scaler.com
[chatgpt](#)